Beginner's Tutorial: The MATLAB Path of the Streaming Range-Doppler SAR Repository

Table of Contents

Beginner's Tutorial: The MATLAB Path of the Streaming Range-Doppler SAR Repository

This tutorial walks through the MATLAB part of this repository from zero. You do **not** need to know SAR or FFT hardware design beforehand. The Simulink model `sys1.slx` is out of scope here.

---

0. The 60-second picture

A **SAR (Synthetic Aperture Radar)** flies past a scene and sends many radar pulses. Each pulse's echo is one column of a 2-D table:

```
            pulse 1  pulse 2  ...  pulse 512     <- "azimuth" (slow time, along the flight path)
sample 1      .         .             .
sample 2      .         .             .
  ...
sample 1024   .         .             .
   ^
   "range" (fast time, distance from the radar)
```

The raw table looks like noise. To turn it into an image of two bright dots (the two targets) we do two **compressions**, each one an FFT, multiply, IFFT sequence:

Step	Direction	Operation	Purpose
1	down each column (range)	FFT	go to frequency domain
2	down each column	multiply by range filter `H_ref`	matched filter for the chirp
3	down each column	IFFT	**range compression done**
4	along each row (azimuth)	FFT	go to Doppler domain
5	along each row	multiply by azimuth filter	matched filter for the Doppler chirp
6	along each row	IFFT	**azimuth compression done** → image

This is the **Range-Doppler Algorithm (RDA)**, minus a refinement called range-cell-migration correction. For these parameters the correction isn't needed (see §3.3).

The repo does steps 1–6 **twice**:

- **Classic path** uses MATLAB's built-in `fft` / `ifft`. This is the reference answer.
- **Proposed (streaming) path** replaces the azimuth FFT/IFFT with hand-written, stage-by-stage FFTs that mimic how an FPGA would do it. It produces the same image, or should (see §6, where I found a catch).

---

1. The file map

File	Role	Type
`init_param.m`	Builds the simulated radar data and all the filters	function
`main.m`	The script you run. Runs the classic path, then the streaming path, then plots	script
`stream_fft.m`	A 512-point FFT made of 9 chained DIF stages	function
`fft_dif.m`	**One** stage of a Decimation-In-Frequency FFT	function
`stream_fft_dit.m`	A 512-point FFT made of 9 chained DIT stages	function
`fft_dit.m`	**One** stage of a Decimation-In-Time FFT	function
`sys1.slx`	Simulink / System Generator model (not covered)	model

Call hierarchy:

```
main.m
 ├── init_param.m                (makes rawData, H_ref, H_az1, H_az2)
 ├── stream_fft.m  ── fft_dif.m  (called 9 times, stage 9 down to 1)
 └── stream_fft_dit.m ─ fft_dit.m (called 9 times, stage 1 up to 9)
```

To run it: open MATLAB in the repo folder, type `main`, press Enter. You need MATLAB plus the **Signal Processing Toolbox** (for `bitrevorder`).

---

2. MATLAB basics you need first

Skim this if you already know MATLAB.

Syntax	Meaning
`clc; clear;`	clear the console and all variables
`%` / `%%`	comment / start of a "section" you can run separately
`A(:,i)`	the whole **column** `i` of matrix `A`
`A(i,:)`	the whole **row** `i` of matrix `A`
`size(A,1)`, `size(A,2)`	number of rows, number of columns
`zeros(size(A))`	pre-allocate a matrix of zeros the same shape as `A`
`1j`	the imaginary unit (radar data is complex)
`.*`	element-by-element multiply (`*` would be matrix multiply)
`.'`	plain transpose (flips rows↔columns, **no** conjugate)
`'`	transpose **with** conjugate
`abs(z)`	magnitude of complex numbers
`[a,b] = f(x)`	call a function that returns two outputs
`function [out1,out2] = name(in)`	defines a function; the file name must match
`fft(v)` / `ifft(V)`	built-in Fourier transform and inverse

Why FFT at all? Convolving a signal with a filter (expensive) equals multiplying their spectra (cheap): `ifft( fft(x) .* H )`. Both compressions use exactly this trick.

---

3. `init_param.m`: building the fake radar data

This function returns four things: `rawData, H_ref, H_az1, H_az2`.

3.1 Parameters (lines 3–14)

Variable	Value	Meaning
`fc`	10 GHz	carrier frequency (X-band)
`c`, `lambda`	3e8, `c/fc` = 3 cm	speed of light, wavelength
`fs`	100 MHz	sample rate of the receiver (ADC)
`prf`	1000 Hz	pulses per second
`Tp`	1 µs	chirp duration
`B`	50 MHz	chirp bandwidth

Variable	Value	Meaning
`K`	`B/Tp` = 5e13	chirp rate (Hz per second)
`v`	100 m/s	platform speed
`numPulses`	512	columns (azimuth)
`numSamples`	1024	rows (range)
`targets`	`[1000 0; 1200 50]`	two point targets: (range 1000 m, azimuth 0 m) and (range 1200 m, azimuth 50 m)

Quick sanity numbers you can derive:

- One range sample = `c/(2*fs)` = **1.5 m** (that's `alphaR`, line 38).
- One pulse spacing along the track = `v/prf` = **0.1 m**.
- A chirp lasts `Tp*fs` = **100 samples**.

3.2 The transmitted chirp (lines 15–17)

```
t = (0:1/fs:Tp-1/fs);
txChirp = exp(1j*pi*K*t.^2);
```

A **chirp** is a pulse whose frequency sweeps linearly. Its phase is $\pi K t^2$. Sweeping a wide band is what lets a radar tell two nearby targets apart after compression.

3.3 Simulating the raw echoes (lines 18–33)

```
for p = 1:numPulses                          % each pulse = one column
    azPos = (p - numPulses/2) * v / prf;     % where the radar is along its path
    for k = 1:size(targets,1)                % each target
        R   = sqrt(targets(k,1)^2 + (azPos - targets(k,2))^2);  % distance radar↔target
        tau = 2 * R / c;                     % round-trip time
        idx = round(tau * fs) + 1;           % which sample the echo starts at
        ...
            phase = exp(-1j * 4 * pi * fc * R / c);   % carrier phase from distance
            rawData(idx:idx+99, p) += phase * txChirp.';  % drop a delayed chirp in
```

In words: for each pulse and each target, compute the distance, convert it to a delay, and place a copy of the chirp at that delay, multiplied by a phase `exp(-j4πfcR/c)`. **That phase changing from pulse to pulse is the Doppler signal.** It is what azimuth compression later exploits. The `if` on line 26 skips echoes that would run off the end of the 1024 samples.

Line 33 adds a little complex Gaussian noise (`0.01*randn`). Because of the noise, **each run produces slightly different data**.

> **Why no range-cell-migration correction?** Over the aperture, `R` for the first target changes by only about `sqrt(1000²+25.6²) − 1000 ≈ 0.33 m`, which is far smaller than one 1.5 m range cell. The echo therefore stays in the same row, so each row can be treated independently. (This is my own check from the numbers, not a claim made in the repo.)

3.4 The range filter `H_ref` (line 35)

```
H_ref = fft(conj(fliplr(txChirp)), numSamples);
```

- `fliplr` reverses the chirp in time, `conj` conjugates it. That is the **matched filter** for the chirp.
- `fft(..., numSamples)` zero-pads it to 1024 points and takes its spectrum.

- The result is a 1×1024 vector. Multiplying a column's spectrum by it is the same as correlating the column with the chirp.

3.5 The azimuth filters `H_az1`, `H_az2` (lines 36–47)

```
fd = (0:numPulses-1) * (prf / numPulses);   % Doppler frequency axis 0 .. ~prf
fd(fd > prf/2) = fd(fd > prf/2) - prf;      % wrap the upper half to negative frequencies
R0 = (1:numSamples) * alphaR;               % range (m) of each row
Ka = -2 * v^2 ./ (lambda * R0);             % Doppler chirp rate; depends on range
h  = exp(-1j * pi * (fd.^2) / Ka(r));       % azimuth matched filter for row r
```

- Along the flight path the Doppler signal is itself a chirp, and its rate `Ka` depends on range, so **every row gets its own filter**. `H_az1` is therefore a 1024×512 matrix.
- `fd` is arranged in the same order MATLAB's `fft` outputs (0, positive, then negative frequencies).

The key difference:

```
H_az1(r,:) = h;                     % natural frequency order  -> classic path
H_az2(r,:) = h(bitrevorder(1:512));  % bit-reversed order       -> streaming path
```

`bitrevorder(1:512)` returns the indices 1..512 in bit-reversed order. Why the streaming path needs this is explained in §5.

---

4. `main.m`, part 1: the classic path (lines 3–35)

```
[rawData,H_ref,H_az1,H_az2] = init_param;   % build everything
```

Lines	Variable	What it does
5–8	`x1`	FFT of every **column** (range FFT)
9–13	`x2`	multiply each column by `H_ref.'` (`.'` turns the 1×1024 row into a 1024×1 column)
14–18	`x3`	IFFT of every column → **range-compressed** data
19–23	`x4`	FFT of every **row** (azimuth FFT)
24–28	`x5`	multiply row `i` by its own filter `H_az1(i,:)`
29–33	`x6`	IFFT of every row → **azimuth-compressed** data
35	`x7 = abs(x6)`	magnitude, which is the image

The pattern to learn: **`for i = 1:size(A,2)` + `A(:,i)` loops over columns; `for i = 1:size(A,1)` + `A(i,:)` loops over rows.** (MATLAB's `fft(A,[],1)` would do all columns at once, but the author loops explicitly to mirror a hardware pipeline.)

5. The streaming path: FFTs you can build in hardware

5.1 Why not just call `fft`?

On an FPGA there is no `fft()` function. You build the FFT from small identical stages ("butterflies"). A 512-point FFT has $\log_2 512 = $ **9 stages**, and data flows through them like water through pipes, which is the "streaming" idea. The repo writes each stage as its own MATLAB function so the Simulink model can be checked against it stage by stage.

5.2 One butterfly stage

Every stage processes pairs of values `(u, v)` that are `m/2` apart and writes two outputs. Here `m = 2^stage`, and `wm = exp(-j2π/m)` generates the **twiddle factors** `w = 1, wm, wm², ….`

`fft_dif.m` (Decimation In Frequency): multiply *after* the add/subtract.

```
y(k+j+1)       = u + v;
y(k+j+1+m/2)   = (u - v) * w;
```

`fft_dit.m` (Decimation In Time): multiply *before* the add/subtract.

```
v = x(k+j+1+m/2) * w;
y(k+j+1)       = u + v;
y(k+j+1+m/2)   = u - v;
```

The loops:

- `for k = 0:m:512-1`: walk through the data in blocks of size `m`.
- `for j = 0:(m/2-1)`: walk through the pairs inside a block.
- `ww(j+1) = w;` records the twiddle values used, so you can compare them with the hardware's twiddle ROM.

Beginner note: the `+1` everywhere is because MATLAB arrays start at index 1, not 0. Also, both functions hard-code `512`, so they only work for 512-point data.

5.3 Chaining stages

`stream_fft.m` calls `fft_dif` for stages **9, 8, 7 … 1**:

```
[y9,w9] = fft_dif(xx,9);   % blocks of 512
[y8,w8] = fft_dif(y9,8);   % blocks of 256
...
[y1,w1] = fft_dif(y2,1);   % blocks of 2
```

`stream_fft_dit.m` does the mirror image, stages **1 → 9**.

5.4 The bit-reversal trick (the clever part)

- A DIF FFT takes input in **natural order** and produces output in **bit-reversed order**.
- A DIT FFT takes input in **bit-reversed order** and produces output in **natural order**.

So if you chain DIF → (point-wise multiply) → DIT, **no reordering memory is needed in between**. That saves a lot of RAM on an FPGA. The only price: the multiplication happens on bit-reversed data, so the filter must be bit-reversed as well. That is exactly why `H_az2 = h(bitrevorder(1:512))` exists.

I verified numerically (Python port of these two files) that the 9 DIF stages produce exactly `fft(x)` in bit-reversed order.

5.5 `main.m`, part 2 (lines 36–68)

Lines	Variable	What it does
38–41	`xx1`	range FFT (same as classic)
42–46	`xx2`	× `H_ref.'` (same)
47–51	`xx3`	range IFFT (same)
52–56	`xx4`	azimuth FFT via `stream_fft` (9 DIF stages), output bit-reversed. The `w1…w9` outputs are the twiddle tables, overwritten each row
57–61	`xx5`	× `H_az2` (bit-reversed filter)
62–66	`xx6`	`stream_fft_dit` (9 DIT stages), back to natural order
68	`xx7 = abs(xx6)`	magnitude

5.6 Plotting (lines 70–73)

```
subplot(2,1,1); mesh(x7)          % classic image, top
subplot(2,1,2); mesh(xx7/512)     % streaming image, bottom, divided by 512
```

The `/512` is there because an un-normalised FFT scales its result by N = 512, while MATLAB's `ifft` includes the `1/N`. Two peaks should appear in each surface plot, one for each target.

---

6. A catch worth knowing about (my finding, please verify on your MATLAB)

The classic path ends with an **IFFT** (`ifft`). The streaming path ends with `fft_dit`, which uses the **forward** twiddle `exp(-j2π/m)`. That is a forward FFT, not an inverse one.

A forward FFT applied where an inverse FFT belongs gives `N · x[(−n) mod N]`. That is the right answer, **scaled by N and index-reversed along azimuth**. Dividing by 512 fixes the scale. Taking `abs` hides the phase, but the reversal remains.

I checked this in a Python port: first on random data with a stand-in filter, then on a port of the full pipeline (`init_param` + `main`), where the flipped comparison matched to a relative error of about 2e-16, versus about 0.59 unflipped:

```
DIF out == bit-reversed FFT:               True
stream |out|/N == |classical|:             False
stream |out|/N == index-reversed classical: True
```

What this means for you:

- The two mesh plots may show the azimuth axis **mirrored** relative to each other. A mirrored image of a target at a different position would look like a shifted target.
- To compare them numerically, flip one of them, e.g. compare `x7(:, [1 512:-1:2])` against `xx7/512`.
- A proper fix would be to conjugate the twiddle (use `exp(+1j*2*pi/m)`) in `fft_dit.m` and divide by 512, or to conjugate the input and output of the last FFT.

Caveat: MATLAB is not installed in my environment, so I did **not** run `main.m` itself; the Python port stands in for it. Run `main` and compare the two plots to confirm.

---

7. Hands-on exercises

1. **Run it.** Type `main`. Both surfaces should show two peaks. Look for them near rows 766 and 900. The echo starts at row ≈ `2*1000/3e8*100e6 + 1` ≈ 668, but the matched filter output peaks about one chirp length (≈ 99 samples) later. Likewise for the second target (starts ≈ 801).
2. **Numerical comparison.** After running, execute:

   ```
   err1 = norm(x7 - xx7/512, 'fro') / norm(x7, 'fro')
   xr   = xx7(:, [1 512:-1:2]) / 512;
   err2 = norm(x7 - xr, 'fro') / norm(x7, 'fro')
   ```

   If §6 is right, `err2` should be far smaller than `err1`.
3. **Test one stage chain alone.**

   ```
   x = randn(1,512) + 1j*randn(1,512);
   [y,~] = stream_fft(x);
   max(abs(y - fft(x)(bitrevorder(1:512))))   % should be ~1e-12
   ```

   (In older MATLAB write it with a temporary variable, since you cannot index a function result directly.)
4. **Remove a target.** Change `targets` in `init_param.m` to a single row. Re-run and watch one peak disappear.
5. **Move a target in range.** Change 1000 to 1100. Does the peak move by about 67 rows? (100 m ÷ 1.5 m.)
6. **Make the streaming path match.** Apply the fix from §6 and rerun exercise 2.
7. **Speed.** Time the classic path versus the streaming path with `tic; ...; toc`. Why is the hand-written one so much slower in MATLAB? (Hint: nested loops, and `y = zeros(1,m)` grows inside the loop in the early stages.)

---

8. Glossary

Term	Plain meaning
Range / fast time	distance from the radar, measured along each received pulse
Azimuth / slow time	position along the flight path, measured pulse to pulse
Chirp	pulse whose frequency sweeps linearly
Matched filter	the filter that gives the strongest output peak for a known signal

Term	Plain meaning
Pulse compression	turning a long chirp echo into a sharp peak
Doppler	frequency shift caused by relative motion; in SAR it varies along the flight path
PRF	pulse repetition frequency, i.e. how often pulses are sent
Butterfly	the basic add/subtract pair operation inside an FFT
Twiddle factor	the complex rotation `exp(-j2πk/m)` used between FFT stages
DIF / DIT	two equivalent FFT structures; one multiplies after, the other before the butterfly
Bit-reversed order	index order obtained by reversing the binary digits of each index (e.g. 1 = 001 → 100 = 4 in 3 bits)
Streaming	data flows through fixed pipeline stages continuously, instead of being processed as whole matrices
RDA	Range-Doppler Algorithm, the SAR focusing method used here

9. One-paragraph summary

`init_param.m` simulates echoes from two point targets and prepares one range filter plus one azimuth filter per range row. `main.m` focuses the data two ways: the classic way, with MATLAB's `fft`/`ifft` on columns then rows, and the streaming way, where the azimuth transform is built from 9 DIF stages followed (after the filter multiply) by 9 DIT stages. Chaining DIF then DIT removes the need for bit-reversal reordering memory, at the cost of a bit-reversed azimuth filter (`H_az2`). The final DIT stage as written is a forward FFT rather than an inverse one, so its output is scaled by 512 and index-reversed in azimuth. Check this when you compare the two plots.